

Previsão de Probabilidade de Acidentes Utilizando Modelos de Aprendizado de Máquina

Aluno Frederico Luiz Strey¹, Orientador Prof. Ph.D. Alexandre Loureiros Rodrigues¹

¹ Especialização em Inteligência Artificial e Ciência de Dados
Universidade Federal do Espírito Santo (UFES)
Universidade Aberta Capixaba (UnAC)

fredericoluizstrey@gmail.com.br, alexandre.rodrigues@ufes.br

Resumo. Com o avanço dos dispositivos de sensoriamento veicular, tornou-se possível coletar e analisar grandes volumes de dados de telemetria, abrangendo informações sobre o comportamento dos condutores, condições dos veículos, localização e contexto ambiental. Este trabalho tem como objetivo desenvolver modelos de aprendizado de máquina capazes de prever a probabilidade de ocorrência de acidentes a partir de dados históricos de direção e características demográficas regionais. Utilizando o conjunto de dados disponibilizado pela competição TCC IA 2025, foram analisadas variáveis relacionadas ao perfil do motorista, padrões de condução e informações temporais, visando identificar fatores de risco e padrões preditivos associados a eventos de acidente. A metodologia inclui etapas de análise exploratória, engenharia de atributos, balanceamento de classes e comparação de diferentes algoritmos supervisionados. Os resultados obtidos contribuem para o aprimoramento de sistemas de monitoramento e prevenção de acidentes, com potencial aplicação em empresas de locação de veículos e em iniciativas voltadas à segurança no trânsito.

Palavras-chave: aprendizado de máquina; predição de acidentes; telemetria veicular; segurança no trânsito; ciência de dados.

1. Introdução

O crescimento das tecnologias de sensoriamento e conectividade em veículos tem possibilitado a coleta em tempo real de informações detalhadas sobre o comportamento de condução, condições mecânicas e contexto de uso dos automóveis. Esses dados, conhecidos como dados de telemetria veicular, tornaram-se essenciais para a análise de padrões de direção, manutenção preventiva e, principalmente, para o desenvolvimento de modelos capazes de prever eventos de risco, como acidentes.

No contexto de empresas de locação de veículos, o monitoramento contínuo das frotas permite compreender melhor o perfil dos condutores e identificar comportamentos que possam aumentar a probabilidade de ocorrência de acidentes. Em especial nos contratos de longa duração, em que os veículos permanecem sob uso prolongado, compreender esses padrões de risco é fundamental para aprimorar políticas de segurança, reduzir custos e prevenir danos materiais e humanos.

O conjunto de dados utilizado neste trabalho foi disponibilizado pela competição TCC IA 2025, que simula um cenário real de análise de telemetria veicular. O dataset contém informações sobre o perfil dos motoristas, características dos veículos, dados

temporais e comportamentais de direção, bem como indicadores demográficos das regiões de circulação. Cada registro representa uma janela de tempo associada a um condutor, acompanhada de uma variável-alvo binária que indica se houve ou não a ocorrência de um acidente na janela seguinte.

O objetivo deste trabalho é desenvolver e avaliar modelos de aprendizado de máquina capazes de estimar a probabilidade de um condutor se envolver em um acidente dentro da janela de tempo futura, com base nas variáveis explicativas fornecidas. Para isso, foram aplicadas técnicas de análise exploratória, engenharia de atributos e modelagem supervisionada, buscando identificar os fatores mais relevantes para a predição de risco e contribuir com soluções práticas para a segurança no trânsito e a gestão de frotas inteligentes.

2. Conjunto de dados

O conjunto de dados utilizado neste trabalho foi disponibilizado pela competição *TCC IA 2025*, hospedada na plataforma Kaggle. O conjunto de dados é composto por informações de telemetria veicular, comportamento de direção, histórico de eventos e características regionais e demográficas, com o objetivo de possibilitar a previsão da probabilidade de ocorrência de acidentes em uma janela de tempo futura.

2.1. Descrição geral

O conjunto de dados consiste em tuplas que representam 157 variáveis (features) para diferentes condutores, observados em múltiplas janelas de tempo. Cada linha contém um identificador único, uma variável-alvo binária (*target*) que indica se ocorreu ou não um acidente na janela temporal subsequente, e um conjunto de variáveis explicativas anonimizadas. Formalmente, cada tupla pode ser representada como $T_i = \{i, target, \{F_1, F_2, \dots, F_{157}\}\}$, onde i identifica o condutor em uma janela de tempo específica, e $target \in \{0, 1\}$ indica a ocorrência de um acidente na janela seguinte.

O número de janelas observadas para cada condutor é variável, e as identificações individuais foram ocultadas no dataset para preservar a privacidade. Além disso, os dados foram previamente pré-processados e anonimizados pela equipe organizadora, de modo que os nomes das variáveis não revelam diretamente seu significado. Assim, o presente trabalho concentra-se na análise experimental de diferentes algoritmos de aprendizado supervisionado, sem a necessidade de realizar etapas adicionais de limpeza ou transformação de dados.

2.2. Estrutura das variáveis

As variáveis explicativas (features) seguem um padrão de nomenclatura que utiliza prefixos e sufixos, conforme descrito abaixo:

- **Prefixos:**

- *cd* — Comportamento de direção: variáveis derivadas de telemetria sobre o modo de condução do veículo em movimento.
- *temp* — Características temporais: informações que descrevem eventos e condições associadas ao período observado.
- *dem* — Características demográficas: informações relacionadas ao perfil do condutor ou do contrato de locação.

- `regiao` — Características espaciais: dados referentes às regiões de circulação do veículo.
- `hist` — Histórico: registros de eventos anteriores envolvendo o condutor ou o veículo.

- **Sufixo:**

- `prep` — indica que a variável passou por algum tipo de pré-processamento, como normalizações, padronizações ou codificações categóricas.

2.3. Arquivos disponibilizados

A competição fornece três arquivos principais:

- `base_treinamento.csv` — conjunto de treinamento, contendo as features e o valor do `target`;
- `base_teste.csv` — conjunto de teste, com as mesmas variáveis explicativas, mas sem o `target`;
- `exemplo_submissao.csv` — arquivo modelo para submissão das previsões geradas pelos participantes.

2.4. Abordagem adotada

Neste trabalho, o foco principal é a avaliação comparativa de diferentes algoritmos de aprendizado de máquina aplicados à tarefa de classificação binária, visando estimar a probabilidade de ocorrência de um acidente na janela de tempo seguinte. Como as etapas de pré-processamento e anonimização já foram previamente realizadas, o estudo se concentra na seleção, treinamento e ajuste de modelos supervisionados, com o objetivo de maximizar o desempenho preditivo.

A métrica de avaliação utilizada é a *Área sob a Curva ROC* (AUC — *Area Under the Curve*) [Pedregosa et al. 2011], que mede a capacidade do modelo em distinguir corretamente entre casos positivos e negativos. Essa métrica é amplamente empregada em problemas de classificação binária e é a adotada oficialmente pela competição.

3. Metodologia Experimental

Esta seção apresenta os modelos de aprendizado supervisionado utilizados para estimar a probabilidade de ocorrência de acidentes, bem como as principais técnicas de ajuste e validação aplicadas. Todos os experimentos foram conduzidos com base no conjunto de dados descrito na Seção 2, utilizando como métrica de desempenho a *Área sob a Curva ROC* (AUC), padrão oficial da competição (Os notebooks com os códigos de experimentação e otimização detalhada estão disponíveis em Notebook de Teste e Comparação de Modelos e Notebook de Treinamento Otimizado).

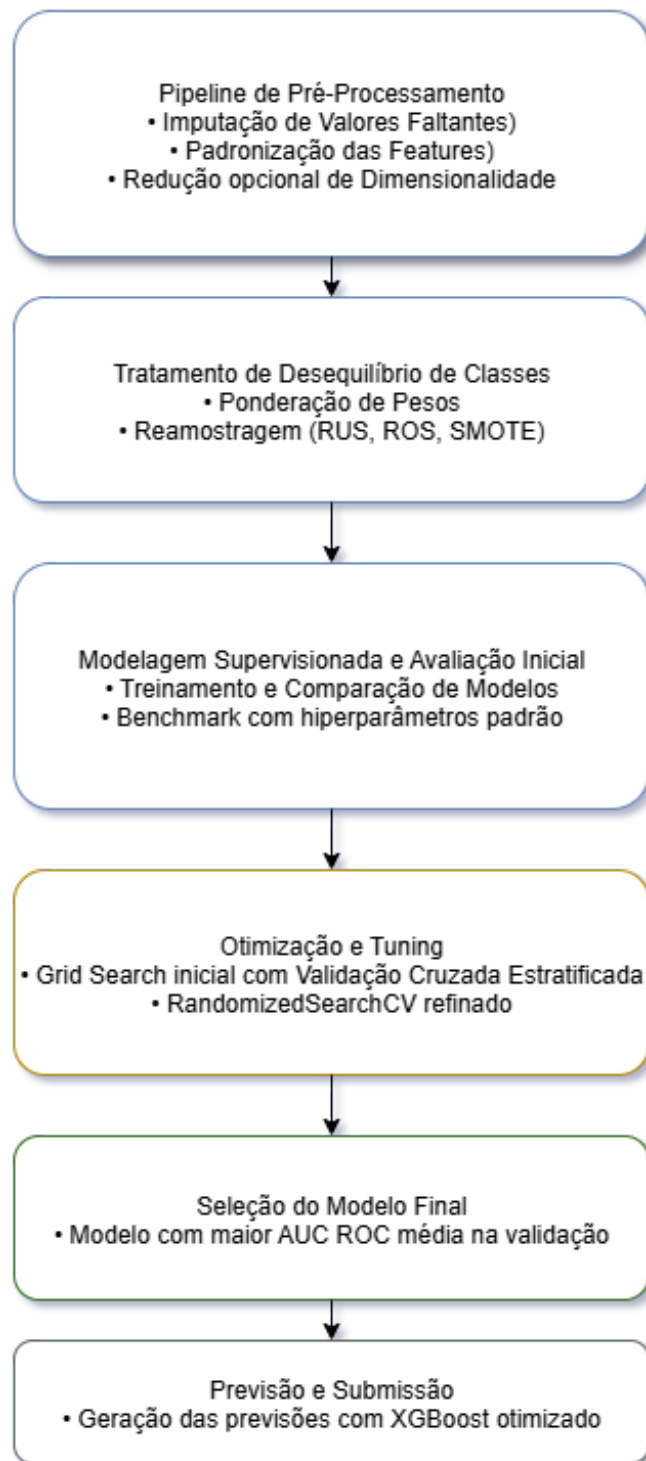


Figura 1. Fluxograma das etapas metodológicas: pré-processamento, balanceamento de classes, modelagem, otimização de hiperparâmetros e seleção do modelo final.

3.1. Modelos Avaliados

Foram testados três algoritmos amplamente utilizados em tarefas de classificação binária com dados tabulares: Regressão Logística [Hosmer et al. 2013], Random Forest

[Breiman 2001], XGBoost [Chen and Guestrin 2016].

- **Regressão Logística:** modelo linear probabilístico. Sua principal vantagem é a interpretabilidade dos coeficientes e a estabilidade em relação a dados ruidosos.
- **Random Forest:** algoritmo baseado em um conjunto de árvores de decisão geradas de forma aleatória e agregadas por votação majoritária. Sua capacidade de capturar interações não-lineares entre as variáveis e resistência a overfitting o tornam um modelo versátil e robusto.
- **XGBoost:** método de *gradient boosting* que constrói árvores de decisão sequenciais, cada uma corrigindo os erros da anterior. O XGBoost é amplamente reconhecido por seu alto desempenho em competições de aprendizado de máquina, devido à eficiência de processamento e aos mecanismos de regularização.

3.2. Técnicas de Treinamento e Validação

Inicialmente, os modelos foram treinados com hiperparâmetros padrão para estabelecer um comparativo preliminar de desempenho. Na etapa seguinte, foram aplicadas técnicas de *tuning* utilizando **Grid Search** e **validação cruzada estratificada (k -fold)** [Pedregosa et al. 2011] para otimização dos principais parâmetros de cada modelo, como profundidade máxima, número de estimadores e taxa de aprendizado.

O **Grid Search** consiste em uma busca exaustiva em uma grade predefinida de combinações de hiperparâmetros. Para cada combinação, o modelo é treinado e avaliado com base em uma métrica de desempenho (como AUC ou acurácia), permitindo identificar o conjunto de parâmetros que maximiza o resultado. Embora computacionalmente custoso, esse método garante uma avaliação sistemática e reproduzível do espaço de hiperparâmetros.

Já a **validação cruzada estratificada (k -fold)** divide o conjunto de dados em k subconjuntos (ou *folds*) de tamanhos semelhantes, preservando a proporção das classes em cada partição. Em cada iteração, um dos subconjuntos é reservado para validação, enquanto os demais são usados para o treinamento. O processo é repetido k vezes, e as métricas obtidas são posteriormente médias, proporcionando uma estimativa mais estável e confiável do desempenho do modelo. Esse método é particularmente útil em bases de dados de tamanho moderado, pois maximiza o uso dos dados disponíveis e reduz o risco de overfitting.

Em algumas execuções do treinamento, o **balanceamento de classes** foi tratado por meio da *ponderação dos pesos de classe* na função de custo dos modelos. Essa estratégia é necessária devido à **desproporção entre as classes** — ou seja, a quantidade de registros de acidentes (classe positiva) é significativamente menor do que a de não-acidentes (classe negativa). A ponderação ajusta o impacto de cada classe no processo de otimização, penalizando de forma mais intensa os erros cometidos na classe minoritária, o que aumenta a sensibilidade do modelo na detecção de casos positivos.

Além dessa abordagem, foram avaliadas técnicas de **reprodução e redução amostral** com o objetivo de testar diferentes formas de mitigação do desequilíbrio. Entre elas, destacam-se:

- **Random UnderSampling (RUS):** reduz aleatoriamente o número de exemplos da classe majoritária, equilibrando as proporções entre as classes, ainda que à custa de perda potencial de informação.

- **Random OverSampling (ROS)**: replica aleatoriamente as amostras da classe minoritária, aumentando sua representatividade sem alterar os dados originais.
- **SMOTE (Synthetic Minority Oversampling Technique)**: gera novas amostras sintéticas da classe minoritária a partir de combinações lineares dos vizinhos mais próximos, criando um balanceamento mais suave e representativo do espaço de características ([Chawla et al. 2002]).

Essas técnicas foram integradas aos *pipelines* de treinamento por meio da biblioteca `imbalanced-learn (imblearn)` [Lemaitre et al. 2017]. Essas abordagens de balanceamento são amplamente recomendadas em problemas de previsão de eventos raros, como falhas, fraudes ou acidentes, pois melhora a capacidade de generalização do modelo e contribui para métricas mais equilibradas, como a *Área sob a Curva ROC* (AUC) e o *F1-Score*.

3.3. Métrica de Avaliação

A métrica utilizada foi a **Área sob a Curva ROC (AUC)**, que mede a capacidade do modelo em distinguir corretamente entre as classes positiva (acidente) e negativa (não acidente). Essa métrica é adequada para cenários de classes desbalanceadas, pois avalia a qualidade da ordenação das probabilidades preditas, e não apenas as classificações binárias.

Os resultados obtidos nas validações indicaram desempenho consistente dos modelos baseados em *boosting* (XGBoost), com AUC superior à Regressão Logística e à Random Forest, evidenciando sua maior capacidade de capturar relações complexas entre as variáveis preditoras e o evento de acidente.

3.4. Otimização dos hiperparâmetros

Após uma etapa exploratória inicial com diferentes algoritmos e técnicas de balanceamento, foi conduzida uma busca sistemática por hiperparâmetros utilizando o método `RandomizedSearchCV`, com validação cruzada estratificada em 3 folds e métrica AUC ROC como critério principal.

Todos os modelos foram integrados em *pipelines* compostos pelas seguintes etapas:

- **Imputação de valores faltantes**: com `SimpleImputer(strategy='median')`, garantindo a consistência dos dados de entrada.
- **Padronização das *features***: por meio do `StandardScaler`, evitando que variáveis com escalas distintas influenciassem desproporcionalmente o treinamento.
- **Redução opcional de dimensionalidade**: via PCA, testando diferentes números de componentes, incluindo a opção de manter os dados originais (`passthrough`).

Para todos os algoritmos utilizados neste trabalho, o balanceamento de classes foi tratado tanto por **ponderação de classes** quanto pela técnica **SMOTE** [Chawla et al. 2002], conforme a natureza do modelo.

No caso do **XGBoost**, os experimentos foram executados com suporte a GPU (`device='cuda'`) e incluíram a otimização de parâmetros como:

- `n_estimators` (número de árvores)
- `max_depth` (profundidade máxima)
- `learning_rate` (taxa de aprendizado)
- `subsample` e `colsample_bytree` (amostragem de linhas e colunas)
- `reg_alpha` e `reg_lambda` (regularizações L1 e L2)
- `scale_pos_weight` (ajuste para a classe minoritária)

Após uma otimização de hiperparâmetros inicial, o XGBoost se destacou e passou por uma **segunda etapa de otimização mais refinada**, com ampliação dos intervalos de busca e ajuste fino de parâmetros críticos. Em paralelo, **Random Forest** e **Logistic Regression** também foram submetidos a processos independentes de otimização, assegurando uma comparação justa entre os modelos.

Por fim, o modelo final foi selecionado com base no **maior valor médio de AUC ROC** obtido na validação cruzada, equilibrando desempenho, estabilidade e tempo de execução. Os resultados obtidos serão discutidos na seção 4.

4. Discussão dos resultados

Nesta seção, apresentamos e discutimos os resultados obtidos nos experimentos conduzidos com diferentes algoritmos e técnicas de balanceamento de classes. Busca-se analisar o comportamento dos modelos frente ao desbalanceamento da base de dados, identificar as métricas mais adequadas para avaliar seu desempenho e justificar a escolha do modelo final utilizado na submissão oficial. Também são destacadas as configurações de hiperparâmetros que contribuíram para o melhor desempenho observado, com ênfase no processo de otimização e na capacidade de generalização do modelo selecionado.

4.1. Análise Inicial dos Modelos e Técnicas de Balanceamento

A partir dos resultados apresentados na Tabela 1, observa-se que os modelos sem técnicas de balanceamento apresentaram *accuracy* artificialmente elevadas (próximas de 0.98), enquanto suas métricas de *recall* e *F1-score* permaneceram próximas de zero. Esse comportamento indica que os classificadores tendem a prever quase todas as amostras como pertencentes à classe majoritária (não-acidente), falhando em identificar os casos da classe minoritária. Após a aplicação de métodos de reamostragem, como *RandomOverSampler*, *RandomUnderSampler* e *SMOTE*, nota-se uma redução significativa na *accuracy*, acompanhada de um aumento expressivo no *recall* e no *F1-score*, refletindo um melhor equilíbrio entre as classes e maior sensibilidade aos casos de acidente. Dessa forma, a utilização de técnicas de balanceamento mostrou-se uma importante ferramenta para permitir que os modelos aprendessem padrões relevantes da classe minoritária e, consequentemente, melhorassem sua capacidade preditiva de forma mais justa e representativa.

Tabela 1. Resumo dos resultados dos modelos com diferentes técnicas de balanceamento.

Model	Balancing	AUC	Accuracy	Precision	Recall	F1
LogisticRegression	No Balancing	0.6679	0.9851	0.0556	0.0003	0.0005
LogisticRegression	RandomOverSampler	0.6673	0.6621	0.0254	0.5836	0.0486
LogisticRegression	RandomUnderSampler	0.6647	0.6479	0.0248	0.5953	0.0476
LogisticRegression	SMOTE	0.6568	0.6454	0.0241	0.5825	0.0463
XGBoost	No Balancing	0.6556	0.8181	0.0311	0.3747	0.0574
RandomForest	RandomUnderSampler	0.6393	0.6610	0.0233	0.5354	0.0446
RandomForest	No Balancing	0.6323	0.9852	0.0000	0.0000	0.0000
RandomForest	RandomOverSampler	0.6314	0.9215	0.0387	0.1805	0.0637
XGBoost	RandomUnderSampler	0.6250	0.0327	0.0149	0.9890	0.0294
XGBoost	SMOTE	0.5878	0.4021	0.0175	0.7146	0.0341
RandomForest	SMOTE	0.5667	0.9098	0.0221	0.1177	0.0371
XGBoost	RandomOverSampler	0.5659	0.0910	0.0149	0.9278	0.0293

O resultado da Tabela 1 demonstra a necessidade de utilizar técnicas de balanceamento e, principalmente, a métrica AUC ROC, que avalia a capacidade de distinção do modelo de forma robusta ao desbalanceamento. Após a aplicação de técnicas de balanceamento, a Accuracy cai para níveis mais realistas, enquanto o Recall para a classe minoritária aumenta consideravelmente, sugerindo que o modelo está aprendendo a identificar os casos de acidentes.

4.2. Otimização de Hiperparâmetros e Escolha do Modelo Final

Após a otimização dos hiperparâmetros, o modelo **XGBoost** foi selecionado para gerar as previsões na base de teste e compor a submissão oficial. A escolha do modelo final considerou, principalmente, a métrica **AUC ROC** obtida durante a validação cruzada, complementada pela análise da estabilidade entre os *folds* e do tempo de execução.

Tabela 2. Resultados do segundo experimento com melhores hiperparâmetros e AUC ROC.

Modelo	AUC ROC (validação)	Melhores Hiperparâmetros
RandomForest	0.6420	max_depth: 11, min_samples_leaf: 3, min_samples_split: 3, n_estimators: 487, pca: passthrough
LogisticRegression	0.6931	C: 0.6605, penalty: l2, solver: lbfgs, pca: passthrough
XGBoost	0.6924	colsample_bytree: 0.8298, gamma: 0, learning_rate: 0.038, max_depth: 3, min_child_weight: 1, n_estimators: 369, reg_alpha: 1, reg_lambda: 5, scale_pos_weight: 1, subsample: 0.6746

Os resultados do segundo experimento indicam que o **XGBoost** apresentou a maior AUC ROC (0.6924) entre os modelos avaliados. A configuração adotada combina baixa profundidade, taxa de aprendizado reduzida e regularização L1/L2, resultando

em um modelo relativamente estável e generalizável. Na base de teste, o modelo obteve um score de 0.6924, refletindo o desempenho alcançado após a otimização dos hiperparâmetros e a estratégia de validação aplicada.

Os hiperparâmetros ajustados revelam um equilíbrio entre simplicidade e regularização, conforme descrito a seguir:

- **max_depth: 3** — Uma profundidade máxima reduzida resulta em árvores rasas, o que ajuda a prevenir *overfitting*, especialmente em bases de dados com alta dimensionalidade ou presença de ruído. Essa escolha favorece a captura de padrões gerais, em vez da memorização de exemplos específicos de treinamento.
- **learning_rate: 0.038** — Uma taxa de aprendizado baixa (≈ 0.04) conduz o modelo a aprender de forma gradual, ajustando-se lentamente a cada nova árvore adicionada. Embora exija um número maior de estimadores (*n_estimators*: 369), essa configuração proporciona maior estabilidade e poder de generalização.
- **reg_alpha: 1** e **reg_lambda: 5** — A presença de regularização L1 e L2 atua penalizando a complexidade excessiva do modelo, reduzindo a chance de *overfitting*. Essa abordagem é especialmente relevante em um *dataset* com 157 *features* anonimizadas, em que redundâncias e correlações podem prejudicar o aprendizado.
- **scale_pos_weight: 1** — O valor neutro desse parâmetro indica que o melhor desempenho foi alcançado sem necessidade de ponderar adicionalmente a classe minoritária. Esse resultado sugere que o balanceamento foi adequadamente tratado nas etapas de validação estratificada e ajuste de hiperparâmetros.

De modo geral, tanto a **Logistic Regression** quanto o **XGBoost** apresentaram resultados competitivos em termos de AUC ROC durante a validação. Entretanto, no teste final, o XGBoost se destacou por apresentar melhor capacidade de generalização, obtendo um score de 0.68752 na submissão oficial.

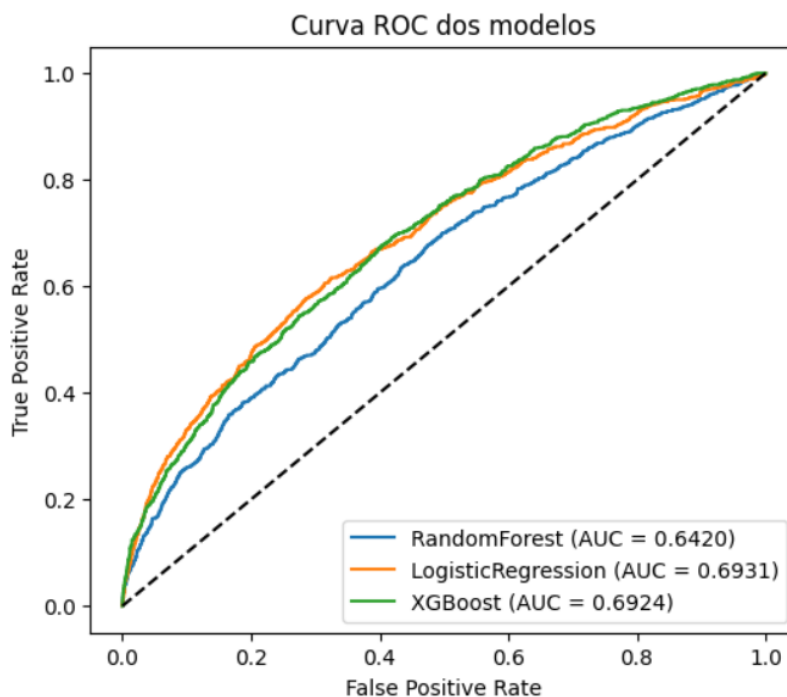


Figura 2. Comparação da métrica AUC ROC entre os diferentes modelos avaliados.

5. Conclusão

Neste trabalho, realizamos uma análise completa de diferentes modelos de classificação aplicados à base de dados fornecida para a competição TCC IA 2025. O caminho percorrido começou com a preparação e divisão dos dados em conjuntos de treino e validação, seguido pela definição e avaliação de múltiplos modelos, incluindo RandomForest, LogisticRegression e XGBoost, com técnicas de balanceamento para lidar com a desproporção de classes.

Os resultados obtidos foram relevantes para análise do desempenho dos modelos. Na validação, o LogisticRegression e o XGBoost apresentaram desempenhos próximos em termos de AUC, embora o XGBoost tenha se destacado ligeiramente na leaderboard do desafio, alcançando um score final de 0.68752. Isso evidencia a robustez do modelo diante de diferentes métricas e configurações, mesmo quando comparado a abordagens lineares mais simples. O RandomForest, embora tenha apresentado desempenho aceitável, ficou abaixo dos outros dois modelos.

Ressalta-se que os recursos computacionais disponíveis foram limitados, o que restringiu a extensão da busca por hiperparâmetros e a possibilidade de testar uma variedade maior de modelos. Assim, utilizamos GridSearch com combinações previamente definidas de hiperparâmetros, mas estudos futuros podem explorar buscas mais robustas, utilizando RandomizedSearch ou outras técnicas de otimização de hiperparâmetros, além da inclusão de outros modelos que possam capturar melhor a complexidade dos dados.

Em síntese, este estudo demonstra que, mesmo com limitações computacionais e conhecimento limitado a cerca da base de dados trabalhada, é possível alcançar resultados

consistentes e significativos, fornecendo uma base sólida para futuras melhorias e experimentos adicionais, seja na otimização de modelos já utilizados, seja na exploração de novas abordagens de machine learning.

6. Referências

Referências

- Breiman, L. (2001). *Random Forests*, volume 45. Machine Learning Journal.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM.
- Hosmer, D. W., Lemeshow, S., and Sturdivant, R. X. (2013). *Applied Logistic Regression*. Wiley, 3rd edition.
- Lemaitre, G., Nogueira, F., and Aridas, C. K. (2017). *imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning*. <https://imbalanced-learn.org/>.
- Pedregosa, F., Varoquaux, G., Gramfort, A., and Michel, V. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.